

Laboratorio de Computación

Salas A y B

Profesor(a): Carlos Chaves Mercado

Asignatura: Fundamentos de programación

Grupo: 8

No de Práctica(s): Práctica de estudio 12: Funciones

Integrante(s): Nolasco Ramírez Sofia

Macay Martínez David

No. de lista o brigada: Mesa 6

Semestre: 2026-2

Fecha de entrega: 15 de mayo de 2026

Observaciones:

CALIFICACIÓN: _____

Objetivo:

El alumnado elaborará programas en lenguaje C que requieran el uso de archivos de texto plano en la resolución de problemas, entendiendo a los archivos como un elemento de almacenamiento secundario.

Introducción:

Los programas del usuario interactúan entre sí a través de la memoria RAM. A pesar de que la información es capturada o procesada desaparece en cuanto el programa finaliza su ejecución. Dentro del campo de la ingeniería, es de gran importancia el uso de elementos de almacenamiento secundario para lograr preservar información, principalmente para el registro de bitácoras o la configuración de sistemas.

El lenguaje de programación C proporciona una solución a este problema mediante el manejo de archivos de texto plano. Un archivo es visto dentro del programa como un flujo secuencial de bytes. Por lo tanto, para interactuar con el almacenamiento secundario, el lenguaje C utiliza una estructura especial llamada file, la cuál actúa como apuntadora que gestiona el canal de comunicación entre el sistema operativo y la aplicación.

En esta práctica se procura el dominio de funciones esenciales para el control de archivos, comprendiendo el ciclo de vida de los datos, desde la apertura del archivo y transferencia de información, hasta el cierre seguro del flujo. Esto permitirá desarrollar programas capaces de almacenar resultados de manera permanente, una herramienta útil para el desarrollo de software profesional.

Desarrollo:

Programa 1.c

Este programa permite abrir un archivo en modo de lectura, de ser posible ya que el archivo debe existir o debe tener permiso de lectura.

```
1 //=====
2
3 Online C Compiler.
4 Code, Compile, Run and Debug C program online.
5 Write your code in this editor and press "Run" button to compile and execute it.
6 =====
7
8 #include<stdio.h>
9 int main()
10 {
11     FILE *archivo;
12     archivo = fopen("archivo.txt", "r");
13
14     if (archivo != NULL)
15     {
16         printf("El archivo se abrió correctamente.\n");
17         int res = fclose(archivo);
18         printf("fclose = %d\n", res);
19     }
20     else
21     {
22         printf("Error al abrir el archivo.\n");
23         printf("El archivo no existe o no se tienen permisos de lectura.\n");
24     }
25     return 0;
26 }
27
28
29
30
```

Error al abrir el archivo.
El archivo no existe o no se tienen permisos de lectura.

...Program finished with exit code 0
Press ENTER to exit console.

Figura 1: Código (abrir cerrar archivo)

Programa 2.c

Este programa permite leer el contenido de un archivo, de ser posible ya que debe existir, a través de la función fgets().

```
1 //=====
2
3 Online C Compiler.
4 Code, Compile, Run and Debug C program online.
5 Write your code in this editor and press "Run" button to compile and execute it.
6 =====
7
8 #include<stdio.h>
9 int main()
10 {
11     FILE *archivo;
12     char caracteres[50];
13     archivo = fopen("gets.txt", "r");
14
15     if (archivo != NULL)
16     {
17         printf("El archivo se abrió correctamente.");
18         printf("\nContenido del archivo:\n");
19         while (fgetc(archivo) != 0)
20         {
21             fgets(caracteres, 50, archivo);
22             printf("%s", caracteres);
23         }
24         fclose(archivo);
25     }
26     return 0;
27 }
28
29
30
31
32
```

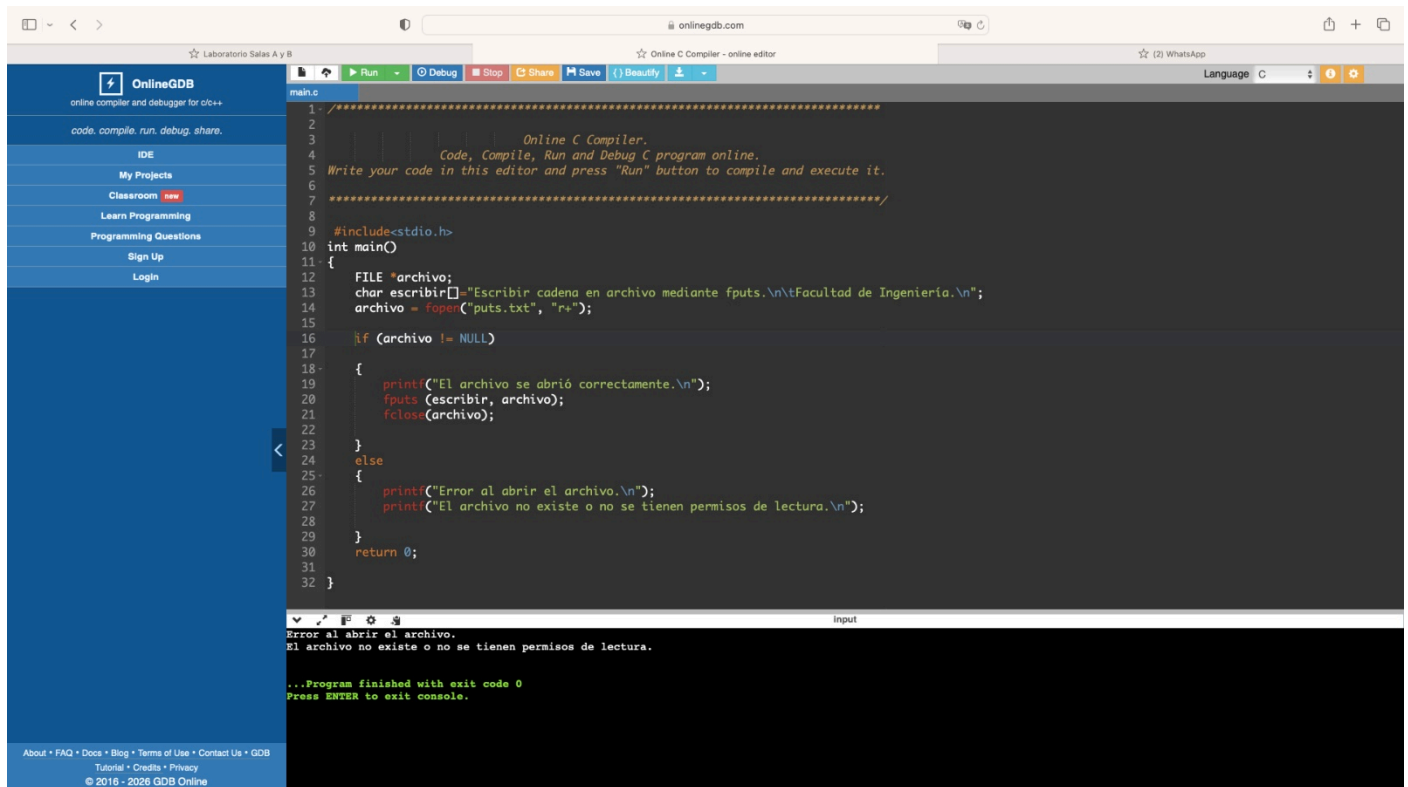
Error al abrir el archivo.
El archivo no existe o no se tienen permisos de lectura.

...Program finished with exit code 0
Press ENTER to exit console.

Figura 2: Código (fgets)

Programa 3.c

Este programa permite escribir una cadena dentro de un archivo, de ser posible ya que debe existir, a través de la función fputs.



```
1 // Online C Compiler.  
2 // Code, Compile, Run and Debug C program online.  
3 // Write your code in this editor and press "Run" button to compile and execute it.  
4  
5  
6  
7  
8  
9 #include<stdio.h>  
10 int main()  
11 {  
12     FILE *archivo;  
13     char escribir[]="Escribir cadena en archivo mediante fputs.\n\tFacultad de Ingenieria.\n";  
14     archivo = fopen("puts.txt", "w+");  
15  
16     if (archivo != NULL)  
17     {  
18         printf("El archivo se abrió correctamente.\n");  
19         fputs (escribir, archivo);  
20         fclose(archivo);  
21     }  
22     else  
23     {  
24         printf("Error al abrir el archivo.\n");  
25         printf("El archivo no existe o no se tienen permisos de lectura.\n");  
26     }  
27     return 0;  
28 }  
29  
30  
31  
32
```

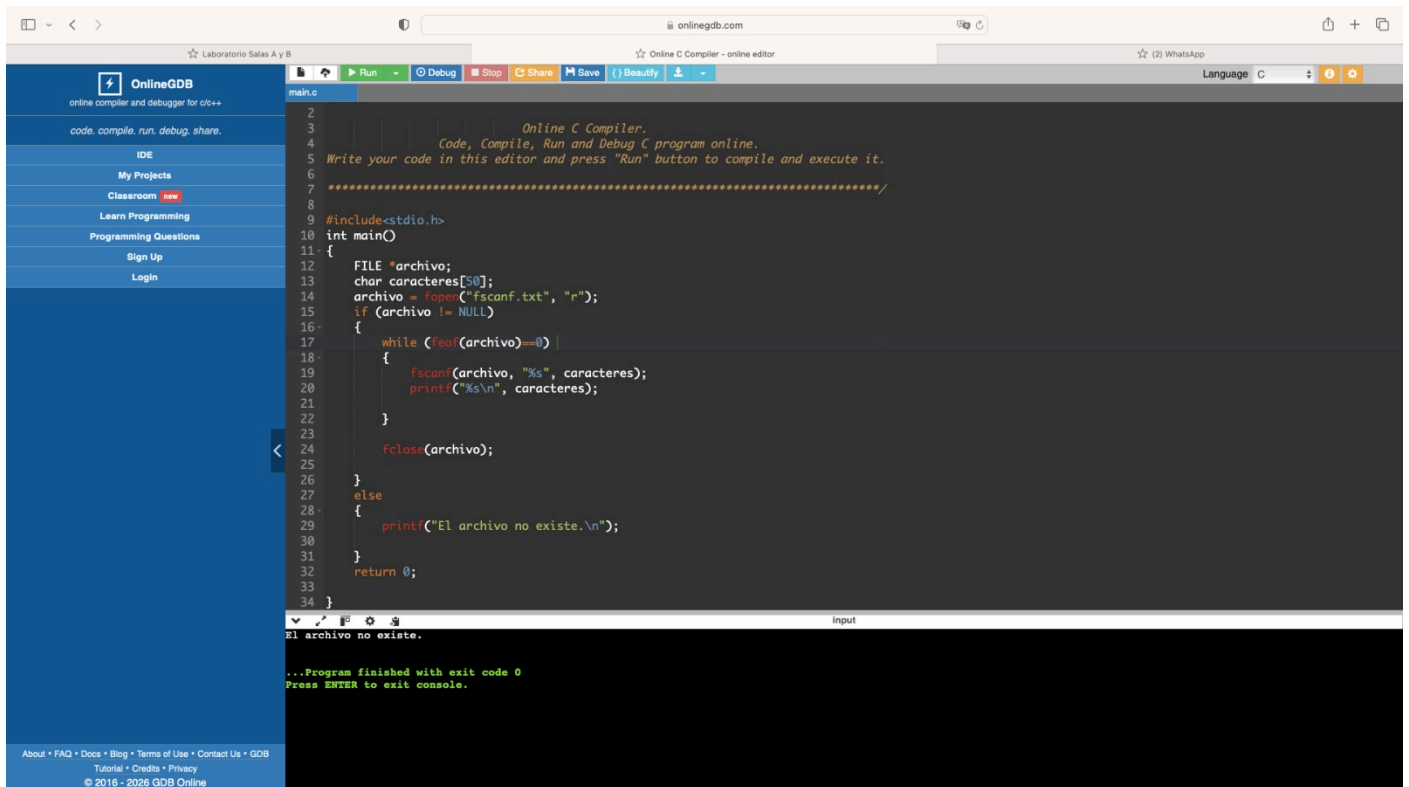
Error al abrir el archivo.
El archivo no existe o no se tienen permisos de lectura.

...Program finished with exit code 0
Press ENTER to exit console.

Figura 3: Código (fputs)

Programa 4.c

Este programa permite leer el contenido de un archivo, de ser posible ya que debe existir, a través de la función fscanf.



The screenshot shows the OnlineGDB web interface. On the left is a blue sidebar with navigation links: OnlineGDB, code.compile.run.debug.share., IDE, My Projects, Classroom (highlighted with a 'new' tag), Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays a C program named 'main.c'. The code includes a header comment, a standard C main function, and logic to open 'fscanf.txt' for reading. It uses a while loop with feof to read and print characters from the file. If the file is not found, it prints an error message. The console at the bottom shows the output: 'El archivo no existe.' and a message indicating the program finished with exit code 0.

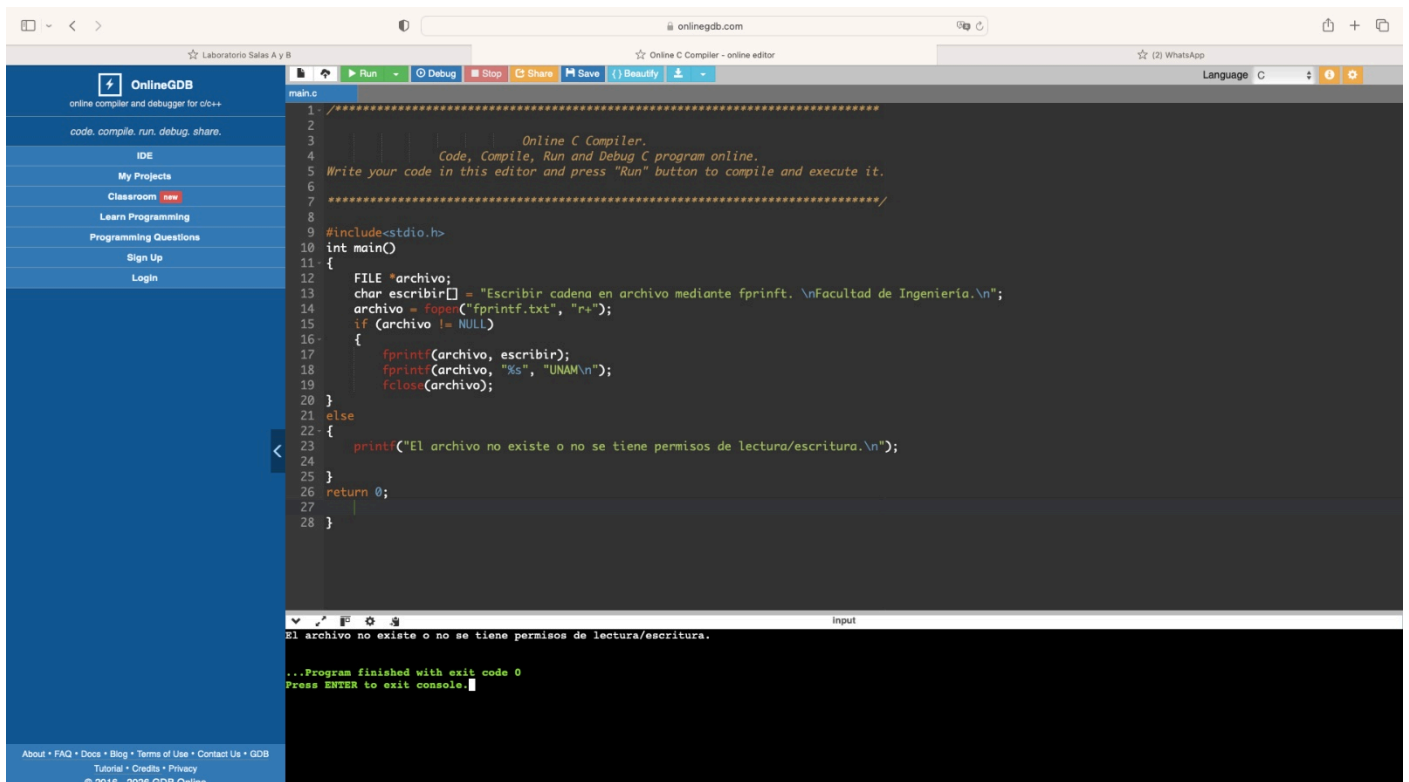
```
1 // Online C Compiler.  
2 // Code, Compile, Run and Debug C program online.  
3 // Write your code in this editor and press "Run" button to compile and execute it.  
4  
5 // =====  
6  
7  
8  
9 #include<stdio.h>  
10 int main()  
11 {  
12     FILE *archivo;  
13     char caracteres[50];  
14     archivo = fopen("fscanf.txt", "r");  
15     if (archivo != NULL)  
16     {  
17         while (feof(archivo)==0) {  
18             fscanf(archivo, "%s", caracteres);  
19             printf("%s\n", caracteres);  
20         }  
21         fclose(archivo);  
22     }  
23     else  
24     {  
25         printf("El archivo no existe.\n");  
26     }  
27     return 0;  
28 }
```

El archivo no existe.
...Program finished with exit code 0
Press ENTER to exit console.

Figura 4: Código (fscanf)

Programa 5.c

Este programa permite escribir dentro de un archivo, de ser posible, a través de la función fprintf.



The screenshot shows the OnlineGDB web interface with a different C program. The code includes a header comment, a standard C main function, and logic to open 'fprintf.txt' for writing. It uses fprintf to write a string and a formatted string to the file, then closes it. If the file is not found or permissions are lacking, it prints an error message. The console at the bottom shows the output: 'El archivo no existe o no se tiene permisos de lectura/escritura.' and a message indicating the program finished with exit code 0.

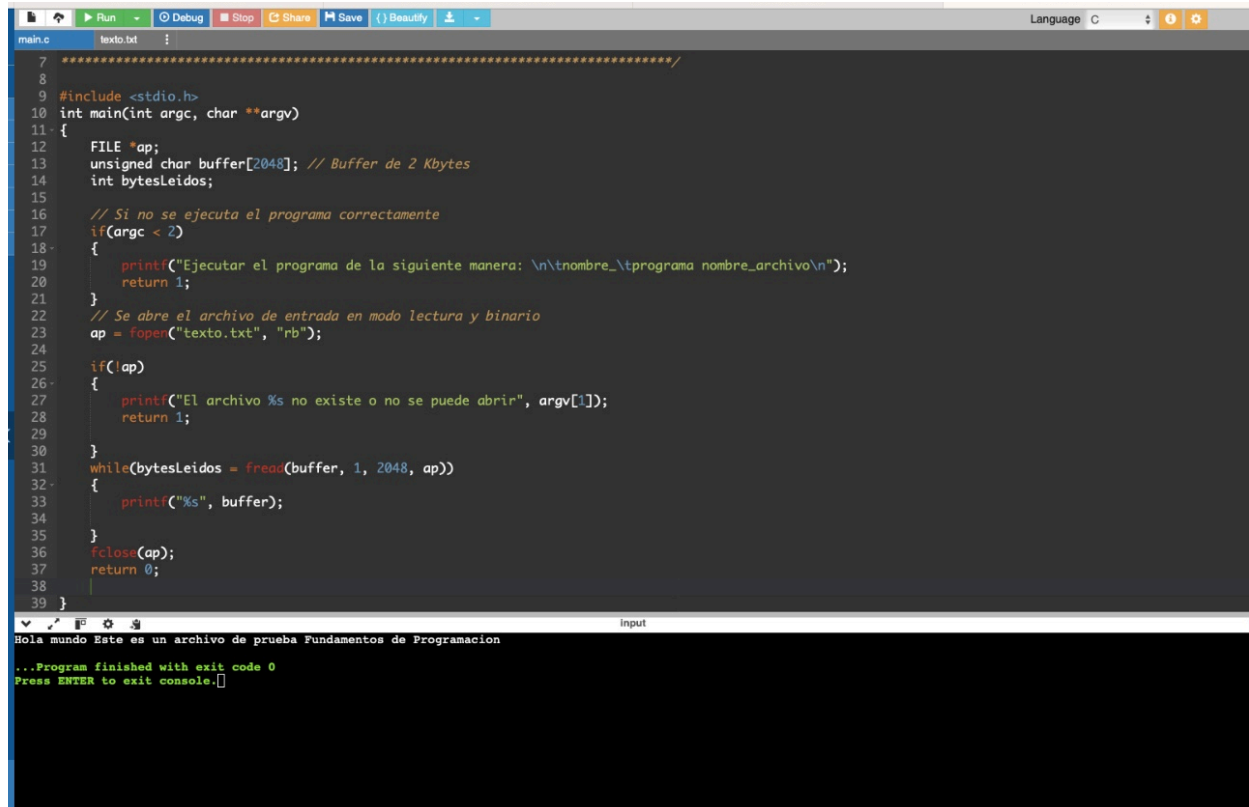
```
1 // Online C Compiler.  
2 // Code, Compile, Run and Debug C program online.  
3 // Write your code in this editor and press "Run" button to compile and execute it.  
4  
5 // =====  
6  
7  
8  
9 #include<stdio.h>  
10 int main()  
11 {  
12     FILE *archivo;  
13     char escribir[] = "Escribir cadena en archivo mediante fprintf. \nFacultad de Ingenieria.\n";  
14     archivo = fopen("fprintf.txt", "w+");  
15     if (archivo != NULL)  
16     {  
17         fprintf(archivo, escribir);  
18         fprintf(archivo, "%s", "UNAM\n");  
19         fclose(archivo);  
20     }  
21     else  
22     {  
23         printf("El archivo no existe o no se tiene permisos de lectura/escritura.\n");  
24     }  
25     return 0;  
26 }
```

El archivo no existe o no se tiene permisos de lectura/escritura.
...Program finished with exit code 0
Press ENTER to exit console.

Figura 5: Código (fprintf)

Programa 6.c

Este programa muestra el contenido de un archivo de texto. El nombre del archivo se recibe como argumento de la función principal.



```
7 ///////////////////////////////////////////////////
8
9 #include <stdio.h>
10 int main(int argc, char **argv)
11 {
12     FILE *ap;
13     unsigned char buffer[2048]; // Buffer de 2 Kbytes
14     int bytesLeidos;
15
16     // Si no se ejecuta el programa correctamente
17     if(argc < 2)
18     {
19         printf("Ejecutar el programa de la siguiente manera: \\n\\nombre_\\tprograma nombre_archivo\\n");
20         return 1;
21     }
22     // Se abre el archivo de entrada en modo lectura y binario
23     ap = fopen("texto.txt", "rb");
24
25     if(!ap)
26     {
27         printf("El archivo %s no existe o no se puede abrir", argv[1]);
28         return 1;
29     }
30     while(bytesLeidos = fread(buffer, 1, 2048, ap))
31     {
32         printf("%s", buffer);
33     }
34     fclose(ap);
35     return 0;
36 }
37
38
39
```

input

Hola mundo Este es un archivo de prueba Fundamentos de Programacion

...Program finished with exit code 0

Press ENTER to exit console.

Figura 6: Código (fread)

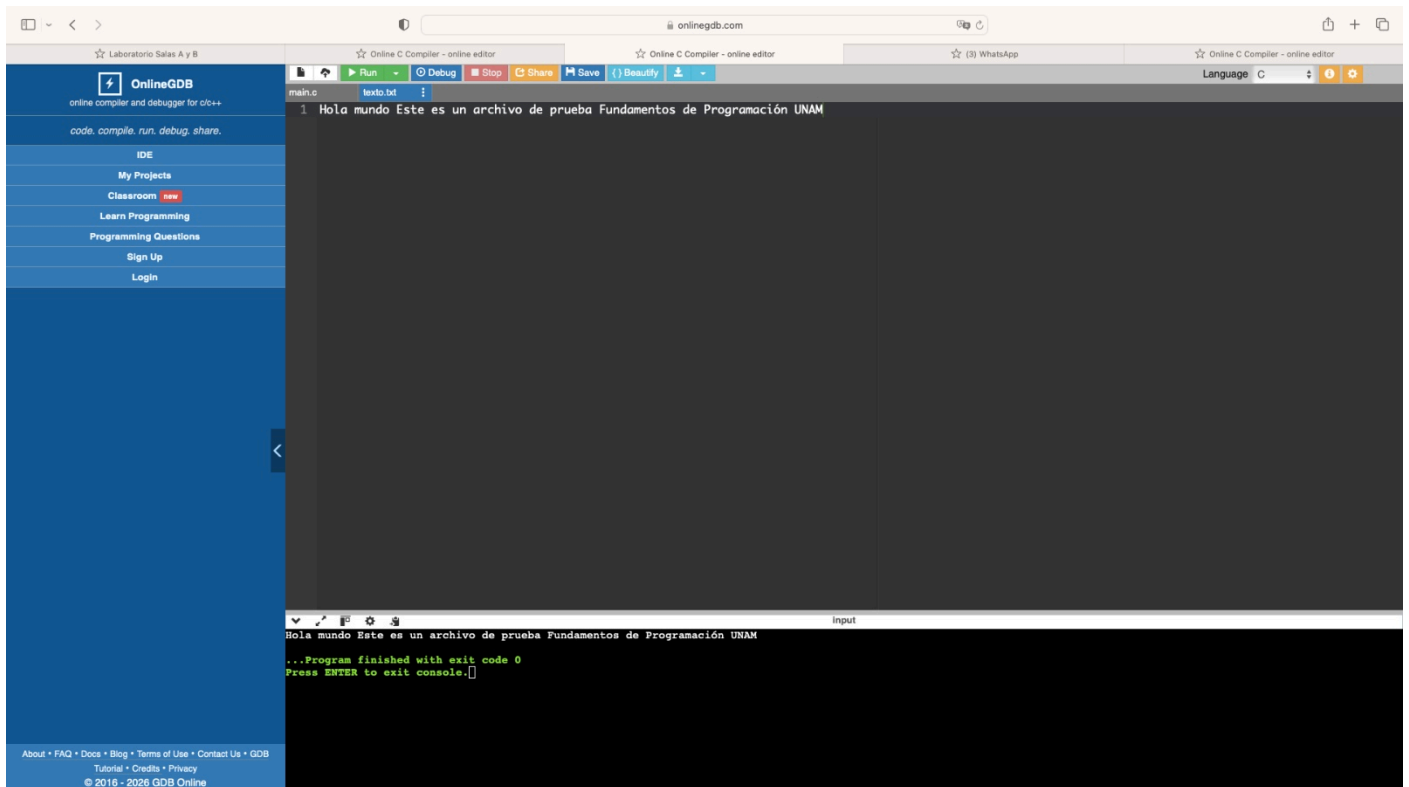
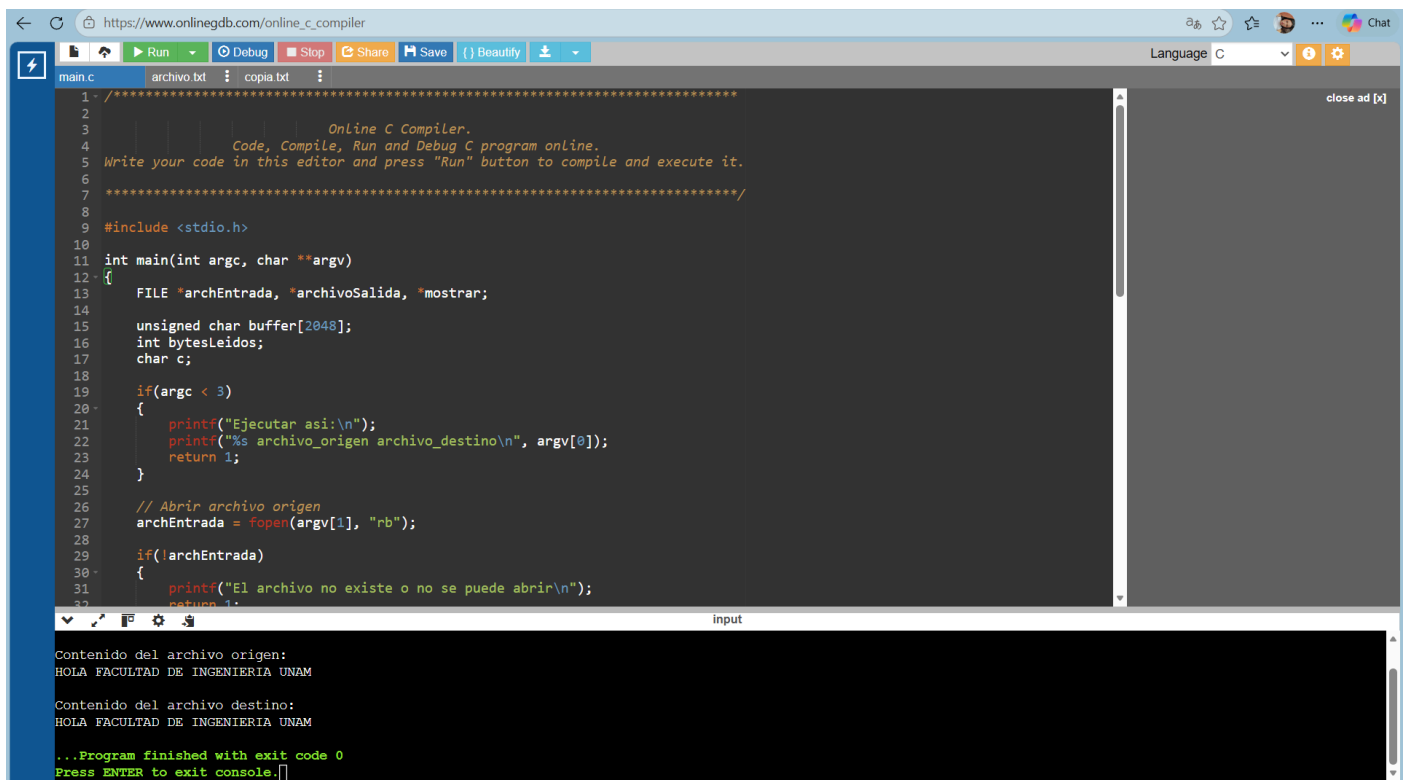


Figura 7: Texto.txt

Programa 7.c

Este programa realiza una copia exacta de dos archivos. Los nombres de los archivos (origen y destino) se reciben como argumentos de la función principal.





The screenshot shows a C++ IDE with a dark theme. The top toolbar includes icons for Run, Debug, Stop, Share, Save, and Beautify. The top right corner shows 'Language: C'. The editor window displays a C++ program for copying a file. The code includes file opening, reading, and writing operations. The console window at the bottom shows the program's execution, displaying the content of the source and destination files, and ending with a success message.

```
main.c | archivo.txt | copia.txt |
49
50     fclose(archEntrada);
51     fclose(archivoSalida);
52
53     printf("\nArchivo copiado correctamente\n");
54
55     // MOSTRAR CONTENIDO DEL ARCHIVO ORIGINAL
56     printf("\nContenido del archivo origen:\n");
57
58     mostrar = fopen(argv[1], "r");
59
60     while((c = fgetc(mostrar)) != EOF)
61     {
62         printf("%c", c);
63     }
64
65     fclose(mostrar);
66
67     // MOSTRAR CONTENIDO DEL ARCHIVO COPIA
68     printf("\n\nContenido del archivo destino:\n");
69
70     mostrar = fopen(argv[2], "r");
71
72     while((c = fgetc(mostrar)) != EOF)
73     {
74         printf("%c", c);
75     }
76
77     fclose(mostrar);
78
79     return 0;
80 }
```

close ad [x]

input

Contenido del archivo origen:
HOLA FACULTAD DE INGENIERIA UNAM

Contenido del archivo destino:
HOLA FACULTAD DE INGENIERIA UNAM

...Program finished with exit code 0
Press ENTER to exit console.

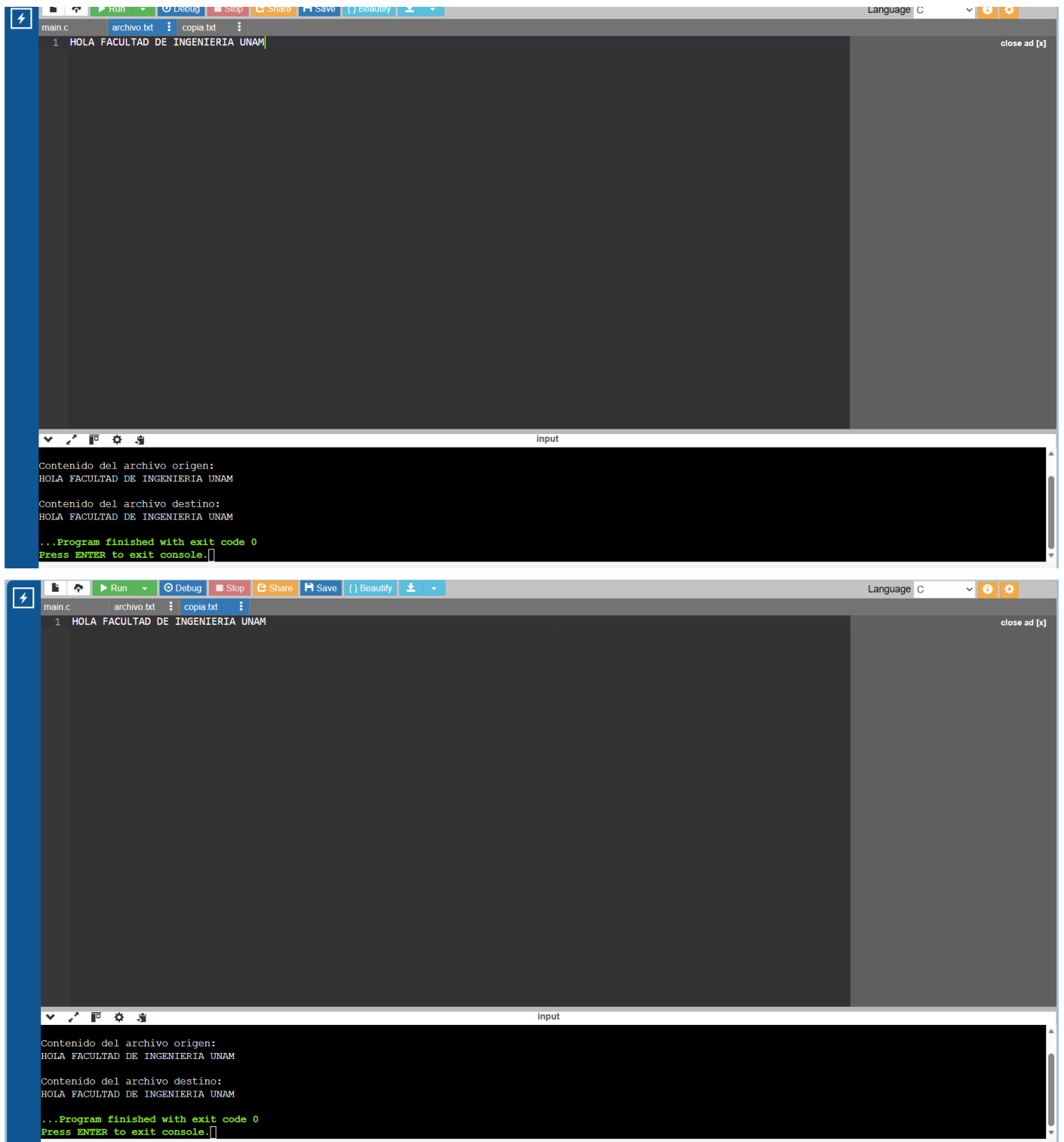


Figura 8 : Código (fwrite)

Análisis de resultados:

Programa 1.c

Este programa tiene como propósito introducir el manejo básico de archivos en lenguaje C mediante el uso de la función `fopen()`. El código intenta abrir un archivo en modo lectura, verificando previamente que el archivo exista y que el sistema tenga permisos suficientes para acceder a él. En caso de que el archivo pueda abrirse correctamente, el programa obtiene un apuntador de tipo `FILE`, el cual funciona como enlace entre el programa y el archivo almacenado en memoria secundaria. Posteriormente, el programa utiliza la función `fclose()` para cerrar el archivo de manera segura y liberar los recursos utilizados por el sistema operativo.

El desarrollo de este programa permite comprender la importancia del manejo adecuado de archivos dentro de una aplicación, ya que cualquier operación de lectura o escritura requiere primero establecer una conexión válida con el archivo. Asimismo, se observa la necesidad de verificar errores durante la apertura para evitar fallos durante la ejecución del programa. Este tipo de procedimientos son fundamentales en aplicaciones que necesitan almacenar información de forma permanente, superando así la volatilidad de la memoria RAM.

Programa 2.c

Este programa tiene como finalidad leer el contenido de un archivo de texto utilizando la función `fgets()`. El archivo debe existir previamente y encontrarse disponible para lectura. La función `fgets()` permite almacenar una línea completa del archivo dentro de una cadena de caracteres, facilitando posteriormente su visualización en pantalla mediante funciones de salida como `printf()`.

El programa demuestra el funcionamiento de la lectura secuencial de archivos de texto, permitiendo recorrer el contenido línea por línea hasta alcanzar el final del archivo. Este método resulta útil cuando se trabaja con documentos extensos, ya que evita cargar toda la información simultáneamente en memoria. Además, el uso de `fgets()` ofrece mayor seguridad en comparación con otras funciones, debido a que permite especificar el tamaño máximo de caracteres que pueden almacenarse en el buffer, reduciendo el riesgo de desbordamiento de memoria.

A través de este programa se comprende cómo un archivo puede ser procesado gradualmente por el sistema, permitiendo extraer información organizada y reutilizarla dentro de un programa. También se refuerza la importancia de cerrar correctamente el archivo después de finalizar la lectura para evitar pérdidas de recursos del sistema.

Programa 3.c

El objetivo principal de este programa es almacenar cadenas de texto dentro de un archivo utilizando la función `fputs()`. El código abre un archivo en modo escritura y posteriormente transfiere una cadena de caracteres desde el programa hacia el archivo de texto. Esta operación permite guardar información de manera permanente en memoria secundaria.

El programa demuestra cómo los datos generados durante la ejecución pueden conservarse incluso después de cerrar la aplicación. La función `fputs()` facilita la escritura de texto completo sin necesidad de manipular carácter por carácter, haciendo más eficiente el proceso de almacenamiento de información textual. Asimismo, el programa evidencia la diferencia entre trabajar con memoria temporal y almacenamiento permanente.

Otra característica importante es que, al abrir el archivo en modo escritura, el contenido previo puede ser reemplazado por la nueva información generada por el programa. Por ello, se observa la relevancia de seleccionar adecuadamente el modo de apertura del archivo dependiendo de la operación deseada. Finalmente, el programa utiliza `fclose()` para asegurar el cierre correcto del archivo y garantizar que toda la información sea guardada correctamente en el dispositivo de almacenamiento.

Programa 4.c

Este programa permite leer datos desde un archivo utilizando la función `fscanf()`, la cual interpreta la información siguiendo un formato específico. A diferencia de funciones como `fgets()`, que trabajan con líneas completas de texto, `fscanf()` permite extraer directamente variables individuales como números enteros, números decimales o cadenas de caracteres.

El programa demuestra cómo un archivo puede ser utilizado como fuente de datos estructurados para una aplicación. La información almacenada en el archivo es procesada y transferida a variables del programa, permitiendo posteriormente realizar cálculos, comparaciones o análisis de datos. Este método resulta especialmente útil en programas que manejan registros, listas o bases de datos simples almacenadas en archivos de texto.

Además, el uso de formatos dentro de `fscanf()` permite controlar el tipo de información que será leída, facilitando la validación y organización de los datos. El programa también refuerza la importancia de verificar que el archivo haya sido abierto correctamente antes de intentar realizar operaciones de lectura. Gracias a este procedimiento, se comprende cómo los archivos pueden funcionar como una fuente permanente de información para distintos programas.

Programa 5.c

Este programa tiene como propósito escribir información dentro de un archivo utilizando la función `fprintf()`. Su funcionamiento es similar al de `printf()`, con la diferencia de que la salida no se muestra en pantalla, sino que se almacena directamente dentro de un archivo de texto.

El programa permite generar archivos con información organizada y estructurada mediante formatos específicos. Esto facilita la creación de reportes, registros de datos, resultados de operaciones matemáticas o cualquier otro tipo de información que necesite conservarse permanentemente. La función `fprintf()` resulta especialmente útil porque permite combinar texto fijo con variables, logrando una salida más clara y ordenada.

A través de este programa se comprende cómo un software puede producir automáticamente archivos de salida con información procesada durante su ejecución. También se observa la importancia de seleccionar el modo adecuado de apertura del archivo para evitar pérdida accidental de información previa. Finalmente, el programa muestra el procedimiento correcto para cerrar el archivo después de concluir la escritura, asegurando que los datos queden almacenados correctamente en memoria secundaria.

Programa 6.c

Este programa tiene como finalidad mostrar el contenido de un archivo utilizando la función `fread()`. El nombre del archivo es recibido como argumento desde la función principal mediante los parámetros `argc` y `argv`, permitiendo al usuario indicar el archivo que desea procesar desde la línea de comandos.

La función `fread()` realiza la lectura del archivo en bloques de bytes almacenados temporalmente dentro de un buffer. Este método ofrece una mayor eficiencia en comparación con la lectura carácter por carácter, especialmente cuando se trabaja con archivos grandes o con información binaria. El programa demuestra el uso de buffers como área temporal de almacenamiento para facilitar la transferencia de datos entre el archivo y la memoria RAM.

Además, este programa permite comprender el manejo de argumentos en la función principal y cómo pueden utilizarse para interactuar dinámicamente con diferentes archivos sin necesidad de modificar el código fuente. También se refuerza la importancia de validar la apertura correcta del archivo antes de iniciar la lectura. Gracias a este procedimiento, el programa puede mostrar en pantalla el contenido completo del archivo indicado por el usuario.

Programa 7.c

Este programa realiza una copia exacta de un archivo mediante el uso combinado de las funciones `fread()` y `fwrite()`. El archivo origen y el archivo destino son proporcionados como argumentos desde la línea de comandos, permitiendo que el programa pueda trabajar con distintos archivos sin necesidad de modificar su estructura interna.

El funcionamiento consiste en leer bloques de datos desde el archivo origen utilizando `fread()`. La información leída es almacenada temporalmente dentro de un buffer y posteriormente transferida al archivo destino mediante `fwrite()`. Este proceso se repite continuamente hasta alcanzar el final del archivo original, logrando así una copia completa y exacta de la información.

El programa demuestra una aplicación práctica del manejo de archivos binarios y del uso eficiente de buffers para optimizar la transferencia de datos. Asimismo, permite comprender cómo los programas pueden manipular grandes volúmenes de información sin necesidad de cargar todo el archivo en memoria al mismo tiempo.

Otra característica importante es el uso de argumentos `argc` y `argv` en la función principal, lo cual brinda mayor flexibilidad al programa al permitir que los nombres de los archivos sean proporcionados durante la ejecución. Finalmente, el programa incluye validaciones para verificar que tanto el archivo origen como el archivo destino puedan abrirse correctamente, evitando errores durante el proceso de copia y garantizando un manejo seguro de los archivos.

Conclusión:

Macay Martínez David:

La realización de esta práctica permitió comprender que el uso de archivos de texto plano cambia el alcance que puede llegar a tener un programa, llegando a ser capaz de interactuar con entornos reales mediante la persistencia de datos. Se logró asimilar el concepto de almacenamiento secundario como un recurso indispensable para superar la volatilidad de la memoria RAM, garantizando que la información procesada no se pierda al concluir la ejecución del software.

Finalmente, se concluye que la manipulación de archivos de texto es una competencia técnica esencial para cualquier ingeniero. Esta capacidad abre las puertas al desarrollo de aplicaciones capaces de realizar lecturas masivas de datos provenientes de sensores, generar reportes automatizados o exportar resultados en formatos compatibles con herramientas de análisis numérico, consolidando así los fundamentos necesarios para el desarrollo de software con aplicaciones industriales y científicas.

Nolasco Ramírez Sofia:

La realización de esta práctica permitió comprender la importancia del manejo de archivos en lenguaje C como una herramienta fundamental para el almacenamiento permanente de información. A través de los distintos programas desarrollados, fue posible identificar el funcionamiento de las principales funciones de lectura y escritura de archivos, tales como `fopen()`, `fclose()`, `fgets()`, `fputs()`, `fscanf()`, `fprintf()`, `fread()` y `fwrite()`, así como la manera en que estas permiten la interacción entre un programa y la memoria secundaria.

Durante el desarrollo de la práctica se observó que el uso de archivos de texto amplía considerablemente las capacidades de un programa, ya que la información procesada puede conservarse incluso después de finalizar la ejecución. Además, se comprendió la importancia de verificar la correcta apertura de los archivos y de realizar un cierre adecuado para evitar errores o pérdida de información.

Asimismo, la práctica permitió reforzar conocimientos relacionados con apuntadores tipo FILE, manejo de buffers y uso de argumentos en la función principal mediante argc y argv. De igual manera, se logró comprender cómo realizar operaciones de lectura, escritura y copia exacta de archivos, procesos ampliamente utilizados en el desarrollo de software profesional.

Finalmente, el manejo de archivos constituye una competencia esencial dentro de la programación, debido a que permite desarrollar aplicaciones capaces de almacenar datos, generar reportes, procesar información y administrar recursos de manera eficiente, siendo una herramienta indispensable en aplicaciones científicas, industriales y tecnológicas.

Bibliografía

El lenguaje de programación C. Brian W. Kernighan, Dennis M. Ritchie, segunda edición, USA, Pearson Educación 1991.